



進行管理をエージェントに任せる。

会議の議事録化・タスク登録・進捗追跡・成果物の版管理・Wiki 更新・決定と成果物の矛盾検知を、人の指示なしで行うエージェントです。

矛盾を見つけたときの通知文（架空データでの実際の出力）

「9/15 の定例で『それ以外の数字は変更しない』と決まっていますが、山田さんが 9/18 に更新した見込み表では商品Aの第3四半期が 1200 → 1500 に変わっています。」

使い方

上段=人がすること 下段=エージェントが自動でやること

利用の流れ（上段=人がすること・下段=エージェントが自動でやること）



解決する状況

- 会議で決めたタスクが登録されていない
- 本人に聞かないとタスクの最新状況が分からない
- _v3_final_修正.xlsx が増え、どれが最新か分からない
- Wiki の手順が古いまま
- 会議で据え置きと決めた数字が翌週の資料で変わっている

デモ手順（人の操作は 2 回）

1. 会議室で話して終了ボタンを押す

決定とタスクが担当者つきで登録される

2. Excel を編集して保存する

版として取り込まれ、差分が担当タスクに紐付く

3. 数秒後に通知が届く

9/15 の決定と食い違っている、と根拠 3 点つきで届く

9/10

評価セットの検知率
誤検知 0

8秒

保存 → 検知
(イベント起動)

-59%

LLM コスト \$0.84
vs 全て high \$2.07

100%

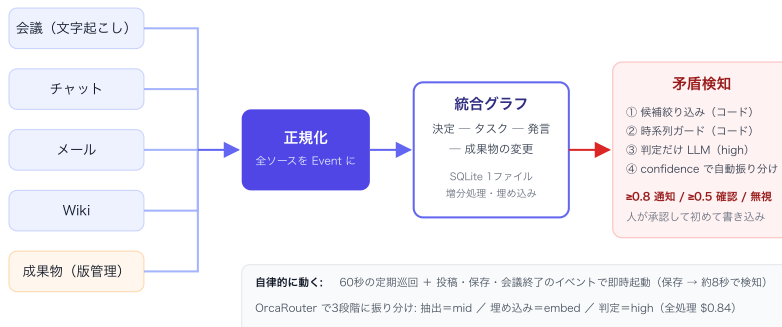
人の指示なしで
完結した判断



GitHub — git clone して3コマンドで動きます

github.com/ryuga-yoshida/ai-hack-2026 / API キー不要の replay モードで検知までを追体験。裏面に仕組みと設計。

仕組み



会議・チャット・メール・Wiki・成果物の 5 つの入力を同じ形式 (Event) に変換して 1 つの DB (SQLite) に保存。成果物が変わるたびに関係する決定を探し出して LLM で判定。60 秒ごとの巡回に加え、投稿・保存・会議終了で即時起動。

OrcaRouter の使い分けと実測

high (Claude Opus)	矛盾の判定だけ。根拠と文脈を渡して 1 件ずつ。\$0.63
mid (Gemini Flash)	決定・タスクの抽出、紐付けの最終段、議事録要約。\$0.21
embed	候補絞り込み・重複統合・紐付け。\$0.001
コード (LLM 不使用)	議事録化・差分・Wiki 生成・時系列ガード・引用検証・閾値判定
合計	\$0.84 (全て high なら \$2.07、削減 59%)

評価

評価セット 10 ケース (矛盾あり 6・許容 4) を毎回再生。**9/10 検知・誤検知 0・見逃し 1** (数式だけの変更)。判定を変えるたびに走らせ、悪化したら戻す。

よくある質問

Q. 誤検知したら？

confidence 0.8 以上だけ通知、0.5~0.8 は確認キュー。却下理由を残し、同種の抑制に使う。判定の内訳 (なぜ疑ったか) も画面で見られる。

Q. チャットや Wiki が自作なのは？

本物の Teams / Confluence は接続に社内承認が要するため。アダプタは実 API (Microsoft Graph 等) の形式で作っており、fetch を差し替えれば判定ロジックは変わらない。

セキュリティ設計

送るデータ	判定に必要な最小限。人名・メール・電話はマスクして送信し応答で復元
指示の混入	議事録本文は指示ではなくデータとして区切って渡す。指示文を混入させた架空データで、抽出に漏れないと画面で確認できる
書き込み	エージェントは通知と提案まで。状態変更・成果物の書き換えは人が承認
幻覚	抽出結果の引用が原文に無ければ機械的に破棄
権限	ロール (admin/member/viewer) と監査ログ。コネクタは読み取り専用
実運用	Teams / Outlook / SharePoint / Confluence はアダプタ差し替え (Graph 形式で空実装済)

Q. 既存ツールとの違いは？

既存の AI は質問に答える型。このエージェントは人が聞かなくても、会議・チャット・成果物を横断して矛盾を探しに行く。

Q. スケールするか？

増分処理と候補絞り込み (埋め込み+語彙ゲート) で、LLM を呼ぶのは絞った後だけ。全件走査はしない。



GitHub



Zenn 記事

Followup / 吉田 龍河 / AI HACK 2026

github.com/ryuga-yoshida/ai-hack-2026 — 設計書・評価セット・LLM キャッシュ同梱。作った経緯と仕組みの解説は Zenn 記事へ。